3522 Bulut Bilişim

Akıllı Şehir Yönetim Platformu

Akıllı Veri Analitiği ve Risk Tahmini Mikroservisi

IoT ve video verilerinden özellik çıkarımı, scikit-learn modeli, rule-based risk motoru ve Lambda inference kapsamı

Alan	Değer
Öğrenci	Ahmet Emre Özcan
Öğrenci No	21290560
Ders	3522 Bulut Bilişim
Öğretim Elemanı	Enver Bağcı
Frontend Repo Linki	https://github.com/eemrezcan/bulutproje-final-frontend
ML Analytics Service Repo Linki	https://github.com/eemrezcan/bulutproje5-ml-analytics-service
Demo Video Linki	https://youtu.be/RbHlxU_dOfk

Kapak proje özeti

Alan	Özet
Kapsam	Proje 3 - Akıllı Veri Analitiği ve Makine Öğrenmesi Uygulaması; IoT ve video verilerinden özellik çıkarımı, scikit-learn modeli, rule-based risk motoru ve Lambda inference kapsamı
AWS odağı	ECS Fargate service, ECR repository, DynamoDB, Lambda inference
Mikroservis sınırı	Ayrı API, ayrı veri modeli, ayrı deployment ve dar IAM yetki sınırı

Özet

Bu rapor, Akıllı Şehir Yönetim Platformu içindeki ML Analytics Service mikroservisini inceler. Servis, IoT ve Video servislerinin REST API'lerinden aldığı son verileri bölge bazında birleştirir; scikit-learn ve joblib tabanlı model artifact'i ile sıcaklık/kalabalık tahmini üretir; rule-based risk motoru ile risk skoru ve karar destek önerisi oluşturur. AWS deploymentta ML API ayrı ECS Fargate servisi olarak çalıştırılmış, düşük maliyetli inference için Lambda alternatifi uygulanmış ve tahminler DynamoDB'deki ml_predictions tablosuna yazılmıştır.

Anahtar Kelimeler: Makine Öğrenmesi, Scikit-learn, Pandas, Joblib, Lambda Inference, DynamoDB, Risk Skoru

İçindekiler

1 Giriş

2 Kapsam

3 Mikroservis sınırı ve sistem mimarisi

4 Teknoloji kararları

5 Veri modeli ve API tasarımı

6 Local çalışma ve demo akışı

7 AWS deployment mimarisi

8 Güvenlik, IAM ve veri sahipliği

9 Test, smoke doğrulama ve kanıtlar

10 Ortak platform entegrasyonu

11 Karşılaşılan sorunlar ve çözüm kararları

12 Sonuç

13 Kaynaklar

1. Giriş

Bu rapor, Akıllı Şehir Yönetim Platformu kapsamında geliştirilen ML Analytics Service bileşenini ayrı bir mikroservis olarak ele alır. Platform tek bir yönetim paneli gibi kullanılsa da her mikroservis kendi veri modeline, kendi API yüzeyine, kendi container image'ına ve kendi AWS yetki sınırına sahiptir.

Temel tasarım hedefi, ders kapsamındaki bulut servislerini yalnızca isim olarak kullanmak değil, doğru problemi doğru AWS servisi ile eşleştirmektir. Bu nedenle raporda local demo akışı ile AWS deployment karşılığı birlikte anlatılmıştır.

2. Kapsam

Bu bölüm, ML Analytics Service mikroservisinin proje içindeki sınırını ve 3522 Bulut Bilişim final yönergesindeki Proje 3 - Akıllı Veri Analitiği ve Makine Öğrenmesi Uygulaması beklentilerini nasıl karşıladığını özetler. Kapsam, yalnızca uygulama ekranını değil; veri giriş kanalını, servis sorumluluğunu, AWS karşılığını ve raporda savunulacak deployment kararlarını birlikte ele alır.

Kapsam maddesi	Projede karşılığı
Akıllı veri analitiği	IoT ve Video servislerinden gelen özet veriler bölge bazında birleştirildi.
Makine öğrenmesi	scikit-learn, pandas ve joblib ile sıcaklık/kalabalık tahmini üretildi.
Karar destek	Rule-based risk motoru ile risk skoru, risk seviyesi ve öneri metni oluşturuldu.
Bulut karşılığı	ML API ECS Fargate üzerinde çalıştı; düşük maliyetli inference için AWS Lambda kullanıldı.
Veri saklama	Tahminler ve risk kayıtları yalnızca ml_predictions DynamoDB tablosunda tutuldu.

Kapsam dışında bırakılan noktalar bilinçli sınırlardır. Amaç üretim ortamı ölçeğinde gerçek şehir altyapısı kurmak değil, ilgili bulut servislerini doğru mimari noktaya yerleştirerek çalışan ve anlatılabilir bir mikroservis sistemi sunmaktır.

3. Mikroservis sınırı ve sistem mimarisi

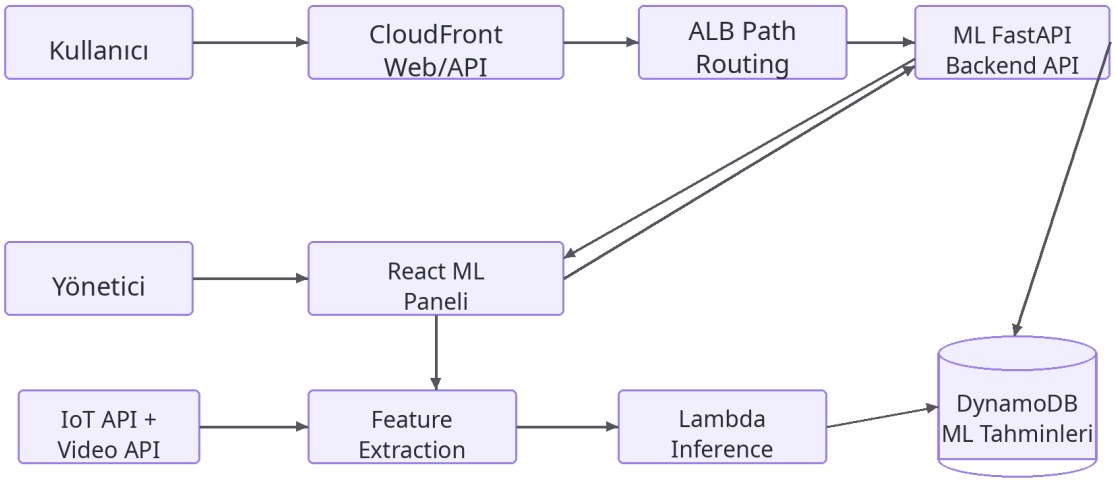
ML Analytics Service aşağıdaki sorumlulukları üstlenir:

- IoT Service API'sinden sıcaklık, nem, hava kalitesi ve trafik verisi alır.
- Video Service API'sinden kişi sayısı, araç sayısı ve kalabalık seviyesi alır.
- Bölge ve zaman bazlı feature extraction yapar.
- Sıcaklık, kalabalık seviyesi, risk skoru ve öneri üretir.
- Tahmin sonuçlarını kendi DynamoDB tablosunda saklar.

Bu servis özellikle şu görevleri üstlenmez:

- Sensör verisi üretmez veya MQTT dinlemez.
- Kamera/video işlemez, OpenCV veya Rekognition çalıştırmaz.
- IoT veya Video servislerinin DynamoDB tablolarına doğrudan bağlanmaz.

Ortak platformda frontend yalnızca REST API çağrıları yapar. Servisler birbirinin DynamoDB tablolarına doğrudan bağlanmaz. Bu yaklaşım mikroservis veri sahipliği ilkesini korur ve deployment ortamında IAM yetkilerinin dar tutulmasını sağlar.



Şekil 1. ML Analytics Service mikroservis mimari diyagramı.

4. Teknoloji kararları

Bileşen	Karar ve gerekçe
Python + FastAPI	Model durumu, eğitim ve tahmin endpointleri için kullanıldı.
scikit-learn	Sıcaklık ve kalabalık tahmini için gerçek ML kütüphanesi olarak kullanıldı.
Pandas + Joblib	Veri hazırlama ve model artifact kaydetme/yükleme için kullanıldı.
Rule-based risk engine	Risk skorunu açıklanabilir ve demo sırasında stabil tutmak için kullanıldı.
Lambda inference	SageMaker maliyeti yerine düşük maliyetli AWS inference alternatifi olarak deploy edildi.
DynamoDB	Bölge bazlı tahmin ve risk eventlerini saklar.

Backend tarafında FastAPI seçimi, hızlı REST API geliştirme, otomatik Swagger dokümantasyonu ve Python ekosistemiyle uyum nedeniyle yapılmıştır. Frontend tarafında React + Vite + TypeScript kullanılmış; canlı görünüm için TanStack Query polling tercih edilmiştir.

5. Veri modeli ve API tasarımı

ml_predictions

Alan	Açıklama
zone	Partition key; tahminin ait olduğu şehir bölgesi.
prediction_timestamp	Sort key; tahminin üretildiği zaman.
prediction_for	Tahminin hedef zamanı; bu projede 60 dakika ufku.
predicted_temperature	Bir saat sonraki sıcaklık tahmini.
predicted_crowd_level	LOW, MEDIUM, HIGH veya CRITICAL kalabalık tahmini.
risk_score / risk_level	0-100 risk skoru ve normal/warning/critical seviyesi.

Alan	Açıklama
recommendation	Operasyon panelinde gösterilecek karar destek önerisi.
input_summary	Tahmine giren IoT ve Video özet verileri.

API yüzeyi

- GET /health
- GET /model/status
- POST /model/train
- POST /predictions/generate
- GET /predictions/latest
- GET /predictions?zone={zone}&limit=50
- GET /zones/{zone}/risk

Endpoint listesi frontend'in ihtiyaç duyduğu okuma, durum ve demo kontrol akışlarını karşılar. Veri üretimi, her modülün kendi doğasına uygun giriş kanalından yapılır.

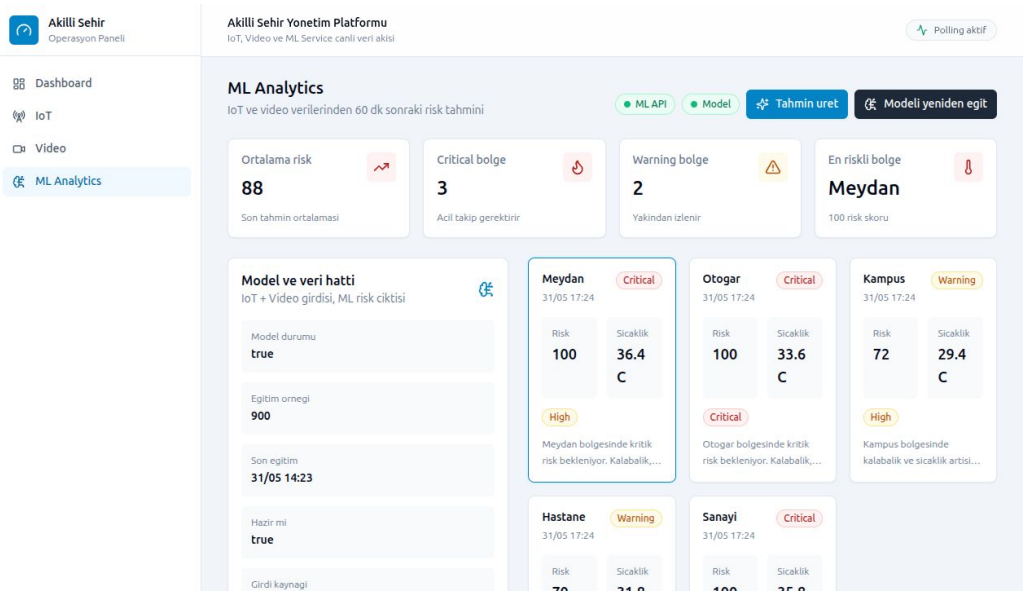
Local-AWS karşılıkları

Local bileşen	AWS karşılığı
Local joblib/scikit-learn model	Lambda inference veya ölçekli alternatif olarak SageMaker endpoint
Local REST veri okuma	CloudFront path routing üzerinden IoT ve Video API okuma
DynamoDB Local	Amazon DynamoDB: ml_predictions
FastAPI ML container	ECS Fargate ML service
Manual prediction üretimi	Lambda invoke veya ML API prediction generate akışı

6. Local çalışma ve demo akışı

Local demo Docker Compose ile çalışacak şekilde tasarlanmıştır. Frontend tek paneldir; IoT, Video ve ML servisleri ayrı container süreçleri olarak ayağa kalkar. DynamoDB Local local veritabanı karşılığıdır.

Akış adımı	Açıklama
Başlangıç	ML Analytics Service
Adım 1	IoT Service REST API
Adım 2	Video Analysis Service REST API
Adım 3	Feature extraction
Adım 4	scikit-learn model + rule-based risk engine
Adım 5	DynamoDB Local: ml_predictions
Adım 6	ML Service REST API
Adım 7	Frontend ML ekranı



Şekil 2. ML Analytics Service canlı deployment ekran görüntüsü.

7. AWS deployment mimarisi

AWS ortamında servis, ayrı ECR repository'si, ayrı ECS Fargate service'i, ayrı task definition ve ayrı IAM task role ile konumlandırılmıştır. Frontend S3 + CloudFront üzerinden yayınlanır; `/api/*` path davranışı ALB'ye yönlendirilir.

AWS akış adımı	Açıklama
Başlangıç	CloudFront / API path routing
Adım 1	ECS ML API
Adım 2	IoT API + Video API REST okuma
Adım 3	Feature extraction
Adım 4	Lambda: smart-city-final-ml-inference
Adım 5	DynamoDB: ml_predictions
Adım 6	Frontend ML Analytics ekranı
Adım 7	Ölçekli alternatif: SageMaker endpoint

AWS kaynağı	Bu rapordaki görevi
ECS Fargate service	smart-city-final-ml-service API endpointlerini çalıştırır.
ECR repository	smart-city-final/ml-service image versiyonlarını saklar.
DynamoDB	ml_predictions tablosunu barındırır.
Lambda inference	smart-city-final-ml-inference beş bölge için tahmin üretip DynamoDB'ye yazar.
CloudFront + ALB	/api/ml path'ini ML target group'una yönlendirir.
SageMaker alternatifi	Daha ölçekli senaryoda joblib/scikit-learn modelinin endpoint olarak sunulabileceği seçenek.

Bu bölümdeki kaynaklar, local Docker Compose mimarisinin buluttaki karşılığıdır. Amaç backendleri yalnızca bir sunucuya koymak değil; veri girişini, analiz işini, saklama katmanını ve API yüzeyini doğru managed AWS servisleriyle ayırmaktır.

Operasyon ve maliyet notları

Konu	Not
Geçici kapatma	ECS service desired-count=0 yapılarak Fargate task maliyeti durdurulabilir.
Veri koruma	DynamoDB, S3, ECR ve IAM kaynakları silinmeden demo verileri korunur.
ALB notu	ALB'nin stop modu yoktur; tamamen maliyeti kesmek için silmek gerekir, demo öncesi önerilmez.
Yaklaşık maliyet	3 Fargate task + ALB + public IPv4 düşük trafikte yaklaşık 0.13 USD/saat düzeyindedir.

8. Güvenlik, IAM ve veri sahipliği

Her mikroservise yalnızca kendi verisine erişim yetkisi verilmiştir. ECS task execution role sadece ECR image çekme ve CloudWatch Logs yazma gibi altyapı görevleri için; uygulama task role ise servis özelindeki DynamoDB/S3/Lambda erişimleri için kullanılır.

Bu ayrım, mikroservis sınırını hem kod hem deployment seviyesinde görünür hale getirir. Bir servisin diğer modülün tablosuna doğrudan erişmemesi, hata izolasyonu ve ilerde ölçekleme açısından daha sağlıklı bir tasarımdır.

9. Test, smoke doğrulama ve kanıtlar

Kontrol	Sonuç
CloudFront ML API health	/api/ml/health status=ok ve model_ready=true döndürdü.
Prediction latest	/api/ml/predictions/latest beş bölge için prediction döndürdü.
Lambda inference	Manuel invoke sonucu ok=true, count=5 olarak doğrulandı.
Frontend smoke	ML Analytics ekranında risk kartları ve beş bölge tahmini görüntülendi.
ECS service	desired=1, running=1, target health=healthy.

Canlı frontend URL'si https://d1k4ey8nhx4dtb.cloudfront.net adresidir.

Bu doğrulamalar teslim videosunda anlatılacak akış için de kanıt niteliğindedir: önce canlı frontend açılır, ardından modül ekranı gösterilir, en son API ve AWS kaynak eşleşmesi açıklanır.

10. Ortak platform entegrasyonu

Bu mikroservis tek başına anlamlı bir modüldür; ancak platform değeri frontend'de diğer servislerle birlikte görünür. IoT verileri şehir durumunu, video analizleri kamera yoğunluğunu, ML servisi ise risk ve öneri katmanını üretir.

Ortak frontend'in veritabanı yoktur. API Gateway kullanılmamış, bunun yerine CloudFront + ALB path routing ile sade ve gösterilebilir bir deployment kurgusu kurulmuştur.

11. Karşılaşılan sorunlar ve çözüm kararları

Sorun / karar alanı	Çözüm
Sentetik veriyle model kararlılığı	Scikit-learn modeli gerçek kullanıldı; risk kararı rule-based motor ile açıklanabilir tutuldu.
Veri sahipliği	ML servisi IoT/Video tablolarını okumadı; REST API ile veri aldı.
SageMaker maliyeti	Demo için Lambda inference seçildi; raporda SageMaker ölçekli alternatif olarak bırakıldı.
Açıklanabilirlik	Risk skoru sıcaklık, trafik, hava kalitesi ve kalabalık göstergeleriyle ilişkilendirildi.

12. Sonuç

ML Analytics Service, Proje 3 kapsamındaki makine öğrenmesi modeli geliştirme, bulutta inference, veriden bilgi çıkarma ve tahmin üretme hedeflerini karşılar. Servis, IoT ve Video modüllerinin ham operasyon verisini karar destek çıktısına dönüştürür ve bu çıktıyı ayrı DynamoDB tablosunda saklar.

Kaynaklar

Kaynaklar: 3522 Bulut Bilişim final proje yönergesi; AWS resmi dokümantasyonu; FastAPI, React/Vite/TanStack Query/Recharts ve scikit-learn, Pandas, Lambda ve SageMaker dokümantasyonları; proje repository dokümanları.